

STUDENT NOTES EXCHANGE PORTAL

A Centralized Academic Document Sharing Platform

Submitted by

AP23110011637 – Balaji Asish

AP23110011605 – Akhil Teja

AP23110010930 – Dharani Shankar

Project Title:

Student Notes Exchange Portal

SyncSpace · May 2026

INTRODUCTION

The Student Notes Exchange Portal — branded as SyncSpace — is an interactive, web-based academic document management platform designed to connect students within the same college, enabling seamless sharing of lecture notes, previous year question papers, lab manuals, and study guides organised by semester, branch, and subject.

In modern academic environments, peer-to-peer sharing of quality study materials remains fragmented across WhatsApp groups, email threads, and informal networks. SyncSpace directly addresses this gap by providing a structured, searchable, and community-driven repository that makes the right notes available to the right student at the right time.

Built on a modern full-stack architecture — React.js (via Vite) on the frontend and Node.js / Express.js with MongoDB on the backend — the portal provides secure user authentication, a filterable notes dashboard, PDF and image file uploads via Cloudinary, a community voting (upvote/downvote) system, and a paginated discovery interface.

Scenario-Based Case Study

Meet Priya, a second-year Computer Science student who frequently struggles to locate notes after missing a lecture. Her batch-mates share files in scattered chat groups where files expire after a few days and subject-wise organisation is non-existent.

Priya's college deploys SyncSpace as part of its digital academic initiative. She registers using her college email, browses notes filtered by Semester 3 → CSE → Data Structures, downloads a high-quality PDF uploaded by a senior and upvotes it. Later, she uploads her own Operating Systems notes — and within a week her file collects twelve upvotes, motivating her to continue contributing.

Thanks to SyncSpace, Priya now has instant access to quality, peer-vetted study material — and her peers benefit from her contributions in return.

SYSTEM REQUIREMENTS

1. Software Requirements

The following tools and platforms are required to develop, test, and run the application:

- Operating System: Windows 10/11, macOS, or Linux — supports cross-platform development.
- Node.js (v20 or above): Provides the runtime environment for the backend server.
- npm (v9 or above): Package manager for React and Express dependencies.
- React.js (via Vite v8): JavaScript library for building the dynamic frontend SPA.
- Express.js (v4): Lightweight REST API framework for the backend.

- MongoDB / MongoDB Atlas: NoSQL database storing users, notes, and votes.
- Mongoose (v8): Schema-based ODM for MongoDB application data.
- Cloudinary: Cloud storage for uploaded PDF and image files.
- JSON Web Tokens (JWT): Secure user authentication and session management.
- Tailwind CSS v3: Utility-first CSS framework for responsive styling.
- Postman: API endpoint testing and debugging during development.
- Visual Studio Code: Preferred IDE with Git integration and terminal support.

2. Hardware Requirements

Minimum and recommended hardware specifications for development and usage:

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 8th Gen+ / Ryzen 5
RAM	8 GB	16 GB
Storage	2 GB free space	10 GB free space (SSD)
Display	1366 × 768	1920 × 1080 or higher
Internet	Required	Broadband (for Cloudinary uploads)

PROJECT ARCHITECTURE

Technical Architecture

The Student Notes Exchange Portal follows a clean client-server architecture with a React.js SPA frontend communicating with a Node.js / Express.js backend over RESTful HTTP APIs. The backend is organised using the Model-View-Controller (MVC) pattern, ensuring separation of concerns across routes, controllers, and Mongoose data models. MongoDB (via Atlas) serves as the primary data store.

The data flow begins with the user interacting with the React frontend built using Vite and structured into reusable components, context providers, pages, and an Axios service layer. When a user registers, logs in, uploads a note, or casts a vote, the frontend makes an authenticated API call to the Express backend.

The backend receives the request through the appropriate route file (authRoutes, notesRoutes, or votesRoutes), validates the JWT token via the auth middleware, then delegates to route handler logic which queries or mutates MongoDB using Mongoose models. JSON responses are returned and the frontend re-renders reactively.

File uploads are handled by Multer with Cloudinary Storage, meaning all uploaded PDFs and images are stored securely in Cloudinary's CDN and only their URLs are persisted in MongoDB.

ER Diagram — Entities & Relationships

The Entity-Relationship diagram maps the core data entities and their associations within the system:

Entities

- **User:** user_id (PK), name, email, password_hash, created_at, updated_at
- **Note:** note_id (PK), title, subject, subject_code, semester, branch, file_url, cloudinary_id, file_type, original_name, uploaded_by (FK→User), upvotes, downvotes, created_at
- **Vote:** vote_id (PK), user_id (FK→User), note_id (FK→Note), vote_type [upvote|downvote], created_at

Relationships

- A User uploads many Notes (1:M).
- A Note is uploaded by exactly one User (M:1).
- A User casts at most one Vote per Note — unique constraint on (user_id, note_id) (M:M resolved via Vote junction).
- Deleting a Note cascades deletion of all associated Votes.

Key Features

- User Registration & Secure Login — JWT-based authentication with bcryptjs password hashing; tokens persist across sessions via localStorage; automatic 401 logout on token expiry.
- Notes Upload with Cloud Storage — PDF and image files up to 20 MB uploaded via Multer and stored permanently on Cloudinary CDN; original filename, file type, and public URL saved to MongoDB.
- Rich Metadata Tagging — every note is tagged with title, subject name, optional subject code (auto-uppercased), semester (1-8), and branch (CSE, ECE, ME, CE, IT, EEE, Other) for precise categorisation.
- Smart Search & Multi-Filter Discovery — real-time debounced search across title, subject, and subject code fields combined with independent dropdown filters for Semester and Branch; active filter pills show applied filters with individual clear buttons.
- Community Voting System — toggleable upvote / downvote mechanism; casting the same vote twice removes it; switching vote adjusts both counters atomically; a unique DB index at (userId, noteId) prevents all duplicate votes.
- Popularity-Ranked Paginated Dashboard — notes sorted by upvote count descending then by recency; 12 cards per page with numbered pagination controls and smooth skeleton loading placeholders.
- Note Detail Page — dedicated view showing full metadata, uploader name and date, net vote score, user current vote state highlighted, Open (new tab) and Download buttons via Cloudinary direct links.
- Owner-Only Delete with Cloud Purge — the note owner can permanently delete their upload; the server removes the file from Cloudinary with correct resource_type for PDFs vs images and cascades Vote document deletion from MongoDB.
- Protected Routes & Auth Guards — PrivateRoute component blocks unauthenticated access to Upload; Axios request interceptor auto-attaches Bearer token; response interceptor auto-logs out and redirects on 401.
- Responsive Dark-Themed UI — Tailwind CSS v3 utility classes with custom colour tokens; Space Grotesk, Manrope, and Inter Google Fonts; Framer Motion page fade-in and card hover animations; mobile-first hamburger navigation.

- Toast Notification System — React Hot Toast provides non-blocking success and error feedback for every user action including login, register, upload, vote, and delete.
- Drag-and-Drop File Upload Zone — the upload page features an interactive drop zone that highlights on drag-over, validates file type and size client-side before submission, and shows a ready-state badge once a file is selected.

ROLES AND RESPONSIBILITIES

Student / Learner (Authenticated User)

- Registration & Account Management — creates a secure account, manages profile, and controls password security.
- Note Browsing — uses search and filters (semester, branch, subject) to discover relevant materials.
- File Upload — submits PDF or image notes with rich metadata (title, subject, subject code, semester, branch).
- Voting — upvotes high-quality notes and downvotes low-quality ones to surface the best content.
- Download & Open — accesses files directly via Cloudinary CDN without server bottlenecks.
- Note Deletion — deletes only notes they uploaded; deletion also removes the Cloudinary file.

Guest (Unauthenticated Visitor)

- Home Page Access — lands on the hero page, reads platform features, views the animated dashboard preview, and clicks Get Started or View Dashboard without any login barrier.
- Full Dashboard Browsing — scrolls through the paginated notes grid and sees all uploaded notes sorted by community vote score; pagination controls work fully without an account.
- Search Without Login — types keywords into the search bar to find notes by title, subject, or subject code; results update in real time with debounce.
- Apply Filters — uses the Semester and Branch dropdown filters and Subject text filter independently or in combination to narrow down the list of notes.
- View Note Detail — clicks any note card to open its full detail page, seeing uploader name, upload date, subject metadata, file type, and the current upvote / downvote score.
- Open Files in Browser — clicks the Open button on the Note Detail page to view a PDF or image file directly in a new browser tab via the Cloudinary CDN link, with no download required.
- Download Files — clicks the Download button to save the note file (PDF or image) directly to their local device from Cloudinary without needing to log in.
- See Vote Scores — can observe the community quality signal (net vote score displayed on every card and detail page) to judge which notes are most valued by peers, even without voting.
- Registration Prompt — when the guest clicks Upload Notes or attempts to vote, the PrivateRoute guard redirects them to the Login page with a clear prompt to create an account.
- No Session State Required — all public browsing, searching, filtering, and downloading work without cookies, tokens, or any stored session data.

Platform Administrator (Future Scope)

- Content Moderation — flag and remove inappropriate uploads.
- Analytics — view aggregate upload/download/vote statistics per subject and semester.
- User Management — suspend or ban abusive accounts.

USER FLOW

Student Flow

Step	Action	Outcome
1	Register / Login	JWT issued; session saved in localStorage
2	Visit Dashboard	Paginated list of notes sorted by score
3	Search & Filter Notes	Real-time filtered results returned from API
4	Open Note Detail	Full metadata + Open / Download buttons
5	Vote (Up / Down)	Score updates instantly; toggle removes vote
6	Upload Notes	File stored on Cloudinary; card appears in feed
7	Delete Own Note	File purged from Cloudinary; votes removed from DB

Guest Flow

- Visits Home → sees hero section with feature highlights and CTA.
- Navigates to Dashboard → browses and searches notes freely.
- Clicks Note Detail → can open / download files.
- Attempts to vote or upload → redirected to Login page.

MVC PATTERN

The Student Notes Exchange Portal strictly follows the Model-View-Controller (MVC) architectural pattern, separating the application into three interconnected layers that provide modularity, maintainability, and scalability.

Model Layer (Data Layer)

Defines data schemas and performs all database operations using Mongoose. The models are:

Model	Schema Purpose
User.js	User accounts: name, email, hashed password (bcryptjs). Includes comparePassword() instance method and password-stripped toJSON().

Model	Schema Purpose
Note.js	Note metadata: title, subject, subjectCode (auto-uppercase), semester enum [1–8], branch enum, fileUrl, cloudinaryId, fileType [pdf image], upvotes, downvotes. Text index on title+subject+subjectCode for search.
Vote.js	Vote records: userId (FK), noteId (FK), voteType [upvote downvote]. Unique compound index on (userId, noteId) prevents duplicate votes.

Controller / Route Layer

Acts as the intermediary between HTTP routes and data models. Routes validate JWT tokens via authMiddleware, execute business logic, and return JSON responses.

Route File	Endpoints	Responsibility
auth.js	POST /register, POST /login, GET /me	User registration, login, JWT generation and verification
notes.js	GET /, GET /:id, POST /, DELETE /:id	CRUD for notes; Multer+Cloudinary upload; search & pagination; owner-only delete with Cloudinary purge
votes.js	POST /:noteId, GET /:noteId/my	Toggle upvote/downvote with atomic note score update; fetch current user's vote

View Layer (React Frontend)

In a REST API architecture, the View is the React SPA. The frontend is built using Vite and organised into reusable components, context providers, pages, and an Axios service layer.

Components (src/components/)

Component	Description
Navbar.jsx	Sticky top navigation with auth-aware links, mobile hamburger menu, user avatar and logout.
NoteCard.jsx	Card rendering note metadata, badges, file type, uploader, date, vote buttons with inline API calls.
NoteFilter.jsx	Semester / Branch / Subject filter row with active filter pills and reset option.
SearchBar.jsx	Debounced search input with clear button.
ProtectedRoute.jsx	Route guard that redirects unauthenticated users to /login.

Pages (src/pages/)

Page	Description
Home.jsx	Landing page with hero, animated dashboard preview, feature highlights and CTA.
Login.jsx	Secure login form with JWT storage in AuthContext and redirect on success.
Register.jsx	Registration form with validation (min 6-char password, confirm match).
Dashboard.jsx	Filterable, searchable, paginated notes grid with skeleton loading states.
NoteDetail.jsx	Full note metadata, voting UI, Open/Download links, owner delete button.
Upload.jsx	Drag-and-drop file zone + metadata form; sends multipart/form-data to the API.

Advantages of MVC in This Project

- Separation of Concerns — each layer has a single, clearly-defined responsibility.
- Scalability — new resource types (e.g., video lectures) can be added via a new model + route without touching existing code.
- Reusability — NoteCard, NoteFilter, and SearchBar are fully reusable across pages.
- Testability — controllers and models can be unit-tested independently.
- Collaboration-Friendly — frontend and backend teams can work in parallel on their respective layers.

PROJECT SETUP AND CONFIGURATION

The project is structured as a monorepo with two sub-folders: client/ (React Vite app) and server/ (Express.js API).

Frontend Setup (React via Vite)

- Navigate to client/ folder in a terminal.
- Run: `npm create vite@latest . -- --template react`
- Select React as the framework and JavaScript as the variant.
- Run `npm install` to install all Vite/React dependencies.
- Install additional packages: `npm install react-router-dom axios framer-motion react-hot-toast react-icons`
- Install dev packages: `npm install -D tailwindcss postcss autoprefixer && npx tailwindcss init -p`
- Run `npm run dev` to start the Vite development server on port 5173.

Backend Setup (Node.js / Express)

- Navigate to server/ folder in a terminal.
- Run `npm init -y` to initialise a Node.js project.

- Install packages: `npm install express mongoose dotenv bcryptjs jsonwebtoken cors multer multer-storage-cloudinary cloudinary`
- Install dev dependency: `npm install -D nodemon`
- Create folder structure: `routes/`, `models/`, `middleware/`, `uploads/`.
- Create server entry point: `index.js` — configures middleware, mounts routes, connects to MongoDB.
- Create `.env` file with: `PORT`, `MONGO_URI`, `JWT_SECRET`, `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`.
- Run `npm run dev (nodemon index.js)` to start the API on port 5000.

Vite Proxy Configuration

Vite's dev server is configured to proxy `/api` and `/uploads` requests to `http://localhost:5000`, eliminating CORS issues during local development.

BACKEND DEVELOPMENT

The backend is built with Node.js and Express.js following the MVC pattern. It provides a complete RESTful API under the `/api` prefix. All write routes are protected by JWT middleware.

Backend Structure

Middleware

File	Responsibility
<code>auth.js</code>	Extracts Bearer token from Authorization header, verifies with jsonwebtoken, attaches <code>req.user</code> ; blocks with 401 on failure.
Global error handler	Catches unhandled errors in the Express pipeline and returns structured JSON error responses.

Routes & API Endpoints

Method	Endpoint	Description
POST	<code>/api/auth/register</code>	Create account; returns JWT + user object
POST	<code>/api/auth/login</code>	Authenticate; returns JWT + user object
GET	<code>/api/auth/me</code>	Return current authenticated user (protected)
GET	<code>/api/notes</code>	List notes with search, filters (semester, branch, subject), pagination
GET	<code>/api/notes/:id</code>	Get single note with populated uploader info
POST	<code>/api/notes</code>	Upload note (multipart/form-data); stores file on Cloudinary (protected)

Method	Endpoint	Description
DELETE	/api/notes/:id	Delete note and Cloudinary file; owner-only (protected)
POST	/api/votes/:noteId	Toggle upvote/downvote; updates note score atomically (protected)
GET	/api/votes/:noteId/my	Return current user's vote on a note (protected)

DATABASE DEVELOPMENT

The platform uses MongoDB as its primary database, accessed through Mongoose for schema definition and data validation. MongoDB Atlas provides a fully managed cloud cluster.

Database Configuration

- Install Mongoose: `npm install mongoose`
- Create `config/db.js` (or inline in `index.js`): connects to `MONGO_URI` from `.env` before starting the Express server.
- Mongoose emits `connected` / `error` events; the server exits on a fatal connection failure.

Schema Definitions

User Schema (`models/User.js`)

- Fields: `name` (String, required, 2–50 chars), `email` (String, unique, lowercase, regex validated), `password` (String, min 6 chars).
- Pre-save hook: `bcrypt.genSalt(10) + bcrypt.hash()` for password hashing.
- Instance method `comparePassword()` for login verification.
- `toJSON()` strips the password field from all API responses.

Note Schema (`models/Note.js`)

- Fields: `title` (max 100), `subject`, `subjectCode` (auto-uppercase), `semester` (enum 1–8), `branch` (enum), `fileUrl`, `cloudinaryId`, `fileType` [`pdf`|`image`], `originalName`, `uploadedBy` (ObjectId ref User), `upvotes`, `downvotes`.
- Virtual: `score = upvotes – downvotes`.
- Compound text index on `title + subject + subjectCode` enables full-text search.

Vote Schema (`models/Vote.js`)

- Fields: `userId` (ObjectId ref User), `noteId` (ObjectId ref Note), `voteType` [`upvote`|`downvote`].
- Unique compound index: `{ userId: 1, noteId: 1 }` — enforces one-vote-per-user-per-note at the database level.
- On toggle: if same vote exists it is deleted (un-vote); if opposite vote exists, both note counters are adjusted atomically.

FRONTEND DEVELOPMENT

The frontend is built with React 19 using Vite 8 as the build tool and Tailwind CSS v3 for styling. It follows a component-based architecture with global auth state managed via React Context API.

Context (src/context/)

- AuthContext.jsx: provides user, token, login(), register(), logout() to the entire app via the Context API. Token and user object are persisted in localStorage.

Services (src/api/)

- axios.js: configures an Axios instance with baseUrl = http://localhost:5000/api, a request interceptor to attach the Bearer token to all outgoing requests, and a response interceptor to handle 401 errors globally (auto-logout + redirect).

Styling & UX

- Dark theme throughout — background #10131a with Tailwind surface-container tokens.
- Typography — Space Grotesk (headlines), Manrope (body), Inter (UI labels) loaded via Google Fonts.
- Framer Motion — page-level fade-in / slide-up animations and micro-interaction hover effects.
- React Hot Toast — toast notifications for all user actions (success, error).
- React Icons (Feather set) — consistent iconography across all components.
- Responsive — mobile-first grid layouts; collapsible hamburger menu on small viewports.

OUTPUT SCREENS — DESCRIPTION

The following screens constitute the complete user interface of the SyncSpace portal:

Screen	Key Elements
Home Page	Hero heading, animated CTA buttons (Get Started / View Dashboard), dashboard preview widget, feature highlights grid, footer.
Login Page	Email + password fields, show/hide password toggle, 'Initiate Session' submit button, link to Register.
Register Page	Full name, email, password, confirm password fields with validation; 'Join SyncSpace' submit button.
Dashboard	Search bar, semester/branch/subject filters, active filter pills, paginated 4-column note grid with skeleton loaders, Upload Note button.
Note Card	Title, subject, file-type badge (PDF/Image), semester + branch + subject code badges, uploader name, date, upvote/downvote buttons with net score.

Screen	Key Elements
Note Detail	Full metadata, Open (external) and Download buttons, voting UI with user's current vote highlighted, owner Delete button, not-logged-in voting nudge.
Upload Page	Drag-and-drop file zone with ready indicator, title/subject/code/semester/branch form fields, 'Complete Upload' button.

CONCLUSION

The Student Notes Exchange Portal — SyncSpace — successfully demonstrates how a modern, full-stack web application can address a real academic challenge: the fragmented and ephemeral sharing of study materials among college students.

By combining React.js, Node.js, Express.js, MongoDB, and Cloudinary into a cohesive platform, the project delivers secure user authentication, scalable file storage, smart search and discovery, and a community-driven quality signal through the voting system.

The adoption of the MVC architecture, RESTful API design, and component-driven frontend ensures the codebase is maintainable, testable, and ready for future extension — such as adding admin moderation, email notifications, or a mobile application powered by the same API.

The SyncSpace project demonstrates strong command of full-stack development principles and provides a tangible, deployable solution that directly benefits the student community.